

UNIVERSIDADE FEDERAL DE PELOTAS
Centro de desenvolvimento tecnológico
Departamento CDTec
Curso de Ciências da computação
Conceitos De Linguagens De Programação



INTERAÇÃO PYTHON & C PARA A IMPLEMENTAÇÃO DE Burning Ship

Lucas Stein
Valtair Augusto

Pelotas, 2026

1. Introdução

Este projeto consiste em uma aplicação gráfica para visualização do conjunto de Burnin ship, utilizando duas linguagens de programação com vocações distintas: Python para a interface com o usuário e C para o processamento numérico intensivo. A motivação principal foi explorar a comunicação entre linguagens de forma prática, sem recorrer a frameworks pesados.

Optamos pelo fractal de Burning Ship porque ele é visualmente interessante e exige um volume considerável de cálculos em ponto flutuante, o que justifica o uso de uma linguagem compilada como o C. A escolha do Python com Tkinter se deu pela rapidez de desenvolvimento e pela facilidade de manipulação de imagens via Pillow.

2. Arquitetura da Aplicação

A aplicação é dividida em dois módulos principais:

Módulo de Cálculo (C): Responsável por percorrer a região do plano complexo, calcular a série de Burnig ship para cada pixel e gerar um buffer de cores no formato RGB. Este módulo é compilado como uma biblioteca dinâmica (.so no Linux, .dll no Windows).

Módulo de Interface (Python): Constrói a janela com Tkinter, oferece campos para ajuste de parâmetros, exibe a imagem e captura cliques do mouse para ampliar regiões. Utiliza a biblioteca ctypes para carregar a biblioteca C e invocar a função de cálculo.

A comunicação entre os módulos se dá exclusivamente pela passagem de um buffer de bytes (unsigned char) alocado no lado Python e preenchido pelo C. Essa abordagem evita cópias desnecessárias e mantém a eficiência.

3. Método de Integração – ctypes

A escolha do ctypes se deu por sua simplicidade e por estar disponível nativamente no Python, sem necessidade de bibliotecas adicionais. O ctypes é uma biblioteca padrão que permite chamar funções escritas em C diretamente do código Python, fazendo a "ponte" entre as duas linguagens.

O processo consiste em:

1. Compilar o código C com flags de posição independente (-fPIC) e gerar uma biblioteca compartilhada.
2. No Python, carregar a biblioteca com ctypes.CDLL.
3. Definir os tipos dos argumentos da função C usando argtypes para garantir a conversão correta.
4. Alocar um buffer de bytes do tamanho adequado ($\text{width} * \text{height} * 3$).
5. Passar o buffer como ponteiro para a função C, que o preenche diretamente.
6. Converter o buffer para uma imagem PIL e exibi-la no Tkinter.

Esse método é eficiente porque o C manipula a memória diretamente, sem intervenção do interpretador Python durante o cálculo, e a transferência de dados é feita por referência, sem cópias.

4. Algoritmo do Fractal Burning Ship

O conjunto de Burning Ship é uma variação do conjunto de Mandelbrot, definido pela iteração da função:

$$z_{\{n+1\}} = (|\operatorname{Re}(z_n)| + i|\operatorname{Im}(z_n)|)^2 + c$$

onde c é um ponto no plano complexo. A diferença fundamental está na aplicação do valor absoluto (fabs) nas partes real e imaginária de z a cada iteração, antes de calcular o próximo valor. Isso gera padrões com formas angulares e detalhes que lembram um navio em chamas, daí o nome "Burning Ship".

O ponto pertence ao conjunto se a sequência não divergir (ou seja, se o módulo de z_n permanecer limitado). Na prática, limitamos o número de iterações e consideramos que o ponto diverge se o módulo ultrapassar 2.

No código C, para cada pixel (px , py), calculamos a coordenada complexa $c = cx + i*cy$ correspondente e executamos o loop de iteração com a aplicação do valor absoluto. O número de iterações é então mapeado para uma cor usando um gradiente pré-definido (função `color_map`). Pontos que atingem o limite máximo de iterações são pintados de preto, representando o conjunto propriamente dito.

5. Instruções de Compilação e Execução

Certifique-se de ter Python 3, Tkinter, Pillow, GCC compatível com a arquitetura do sistema operacional.

Antes de executar o programa instale a biblioteca: `pip install pillow`

No terminal, na raiz do projeto, execute: `gcc -Wall -O3 -shared -o src/libmandel.dll src/Burnig Ship.c`

Execute `python gui/main.py`.

A janela será aberta com o fractal já renderizado. Use os campos para ajustar parâmetros e clique na imagem para ampliar.

6. Dificuldades Encontradas e Soluções

Alinhamento de memória: No início, o buffer retornado pelo C não era interpretado corretamente pelo PIL. Descobrimos que o `Image.frombuffer` espera que os dados estejam em ordem RGB contínua; ajustei a indexação no C para escrever sequencialmente.

Caminho da biblioteca: Em sistemas Windows, a extensão é .dll; no Linux, .so. Adicionei uma verificação no Python para tentar ambos os caminhos.

Inversão do eixo Y: As coordenadas de tela têm a origem no canto superior esquerdo, enquanto o plano complexo tem o eixo Y crescente para cima. Fizemos a conversão adequada no método `on_click`.

7. Conclusão

O projeto atingiu os objetivos propostos: integrar duas linguagens de forma limpa e eficiente, com uma interface utilizável. A separação entre cálculo e apresentação facilita futuras modificações por exemplo, substituir a GUI por uma web ou adicionar paralelismo com OpenMP no código C.